

Experiment # 08

Process related System Calls in Linux

Objective

In this lab we will learn about system call techniques i.e.

- fork()
- exit()
- wait()
- exec()

Software Tools

- Ubuntu 16.04
- GCC Compiler
- Online Compiler https://www.onlinegdb.com/online_c_compiler

Theory

System Call:

A system call is just what its name implies—a request for the operating system to do something on behalf of the user's program.

1. fork()
2. wait()
3. exec()
4. exit()

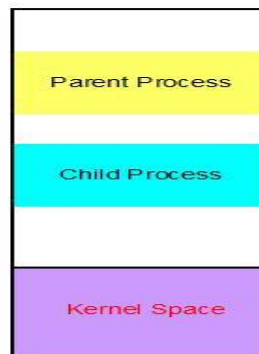
fork()

1. When the fork system call is executed, a new process is created which consists of a copy of the address space of the parent.
2. The return code for fork is zero for the child process and the process identifier of child is returned to the parent process.
3. On success, both processes continue execution at the instruction after the fork call.
4. On failure, -1 is returned to the parent process.

fork()—Sample Code

```
main()
{
    int pid;

    pid = fork();
    if (pid == 0) {
        /* Code for child */
    }
    else {
        /* Code for parent */
    }
}
```



Implementing `fork()` system call using C program**Example 1**

```
#include <stdio.h>
#include <unistd.h>
int main(void)
{
    pid_t pid;
    pid = fork();
    printf("hello");
    return 0;
}
```

Example 2

```
#include<stdio.h>
#include <unistd.h> /* contains prototype for fork */
#include <sys/types.h>

void main()
{
    pid_t pid;
    pid =fork();
    if (pid ==0)
        printf("\n I'm the child process");
    else if (pid> 0)
        printf("\n I'm the parent process. My child pid is %d",pid);
    else
        perror("error in fork");
}
/*
```

Note: `pid_t` is a data type defines in `types.h`.

1. `pid_t` is a signed integer type only. Then why should we use `pid_t`?

A: We don't have to know what type of integer (an int, short int, or long int)the implementation uses for a pid value. So we use `pid_t`.

The `perror()` function is C's error-handling tools. When called, `perror()` displays a message on `stderr`(display screen) describing the most recent error that occurred during a library function call or system call.

*/

Example 3

```
#include <stdio.h>
#include<unistd.h>
```

```
int main(void)
{
```

```
printf("Hello World!\n");
pid_t pid=fork( );
printf("\n%d hello1",pid);
printf("\n%d hello2",pid);
}
```

wait()

- The wait system call suspends the calling process until one of its immediate children terminates.
- If the call is successful, the process ID of the terminating child is returned.
- **Zombie process**—a process that has terminated but whose exit status has not yet been received by its parent process or by in it.
- If a parent dies before its child, the child (orphan process) is automatically adopted by the original —init process whose PID is 1.
- **wait(int *status);**
- where status is an integer value where the UNIX system stores the value returned by child process.

Implementing wait system call using C program

Example#4

```
#include <stdio.h>
#include<unistd.h>
#include<sys/wait.h> /* contains prototype for wait */
#include<stdlib.h>
int main(void)
{
    pid_t pid, status;
    pid=fork();
    if(pid==-1)    /* check for error in fork */
    {
        printf("fork failed\n");
        exit(1); /* exit ( ) terminates the process which calls this function and returns the exit status
        value.
        exit(1) for abnormal termination (when error occurs) exit(0) for normal termination */
    }
    if(pid==0)
    { /* Child */
        printf("Child here!\n");
    }
    else
    { /* Parent */
        wait(&status); /* parent waits for child to finish */
        printf("Well done kid!\n");
    }
    return 0;
}
```

```
}
```

Example#5

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main (void)
{
    pid_t forkresult ;
    printf ("%d: I am the parent. Remember my number!\n", getpid( ) ) ;
    printf ("%d: I am now going to fork ... \n", getpid( ) ) ;
    forkresult = fork ( ) ;
    if (forkresult != 0)
    {
        /* the parent will execute this code */
        printf ("%d: My child's pid is %d\n", getpid( ), forkresult ) ;
    }
    else /* forkresult == 0 */
    {
        /* the child will execute this code */
        printf ("%d: Hi! I am the child.\n", getpid( ) ) ;
    }
    printf ("%d: like father like son. \n", getpid( ) ) ;
    return 0;
}
```

exec()

- Typically the exec system call is used after a fork system call by one of the two processes to replace the process' memory space with a new executable program.
- The new process image is constructed from an ordinary, executable file.
- There can be no return from a successful exec because the calling process image is overlaid by the new process image.
- **execl (const char *file, const char *arg0, ..., const char *argn, char *)0);**

execl Takes the path name of an executable program (binary file) as its first argument. The rest of the arguments are a list of command line arguments to the new program argv[] The list is terminated with a null pointer.

Implementing exec() system using C program

The exec family has many functions in C. These C functions are basically used to run a system command in a separate process that the main program and print the output.

In this article, I am going to talk about the exec family of functions and show you how to use each one of these exec family function in C. So, let's get started.

C System Functions in Exec Family:

The exec function families are defined in the header **unistd.h**. So, you must use this header on the C program where you want to use these functions.

The available exec functions along with their function parameters are given below:

1. `int execl(const char *path, const char *arg, ..., NULL);`
2. `int execlp(const char *file, const char *arg, ..., NULL );`
3. `int execlv(const char *path, char *const argv[]);`
4. `int execlvp(const char *file, char *const argv[]);`
5. `int execlenv(const char *path, const char *arg, ..., NULL, char * const envp[] );`
6. `int execlve(const char *file, char *const argv[], char *const envp[]);`

1. execl() System Function:

In `execl()` system function takes the path of the executable binary file (i.e. **/bin/ls**) as the first and second argument. Then, the arguments (i.e. **-lh**, **/home**) that you want to pass to the executable followed by **NULL**. Then `execl()` system function runs the command and prints the output. If any error occurs, then `execl()` returns -1. Otherwise, it returns nothing.

Syntax:

```
int execl(const char *path, const char *arg, ..., NULL);
```

An example of the `execl()` system function is given below:

```
#include <unistd.h>

int main(void) {
    char *binaryPath = "/bin/ls";
    char *arg1 = "-lh";
    char *arg2 = "/home";

    execl(binaryPath, binaryPath, arg1, arg2, NULL);

    return 0;
}
```

2. execlp() System Function:

`execl()` does not use the **PATH** environment variable. So, the full path of the executable file is required to run it with `execl()`. `execlp()` uses the **PATH** environment variable. So, if an executable file or command is available in the **PATH**, then the command or the filename is enough to run it, the full path is not needed.

Syntax:

```
int execlp(const char *file, const char *arg, ..., NULL );
```

We can rewrite the `execl()` example using the `execlp()` system function as follows:

```
#include <unistd.h>

int main(void) {
    Department of Computer Engineering
    HITEC University Taxila
```

```

char *programName = "ls";
char *arg1 = "-lh";
char *arg2 = "/home";

execlp(programName, programName, arg1, arg2, NULL);

return 0;
}

```

3. **execv() System Function:**

In execl() function, the parameters of the executable file is passed to the function as different arguments. With execv(), you can pass all the parameters in a NULL terminated array **argv**. The first element of the array should be the path of the executable file. Otherwise, execv() function works just as execl() function.

Syntax:

```
int execv(const char *path, char *const argv[]);
```

We can rewrite the execl() example as follows:

```
#include <unistd.h>
```

```

int main(void) {
    char *binaryPath = "/bin/ls";
    char *args[] = {binaryPath, "-lh", "/home", NULL};

    execv(binaryPath, args);

    return 0;
}

```

4. **execvp() System Function:**

Works the same way as execv() system function. But, the PATH environment variable is used. So, the full path of the executable file is not required just as in execlp().

Syntax:

```
int execvp(const char *file, char *const argv[]);
```

We can rewrite the execv() example as follows:

```
#include <unistd.h>
```

```

int main(void) {
    char *programName = "ls";
    char *args[] = {programName, "-lh", "/home", NULL};

    execvp(programName, args);

    return 0;
}

```

5. **execle() System Function:**

Works just like `execl()` but you can provide your own environment variables along with it. The environment variables are passed as an array **envp**. The last element of the **envp** array should be `NULL`. All the other elements contain the key-value pairs as string.

Syntax:

```
int execle(const char *path, const char *arg, ..., NULL, char * const envp[] );
```

An example of the `execle()` system function is given below:

```
#include <unistd.h>
```

```
int main(void) {
    char *binaryPath = "/bin/bash";
    char *arg1 = "-c";
    char *arg2 = "echo `Visit $HOSTNAME:$PORT`";
    char *const env[] = {"HOSTNAME=www.linuxhint.com", "PORT=8080", NULL};

    execle(binaryPath, binaryPath, arg1, arg2, NULL, env);

    return 0;
}
```

```
#include <unistd.h> /* for 'execle' */
#include <stdio.h> /* for 'printf' and 'NULL' */
#include <stdlib.h> /* for 'errno' */
#include <errno.h> /* for 'ENOENT' and 'ENOMEM' */
```

```
char *env[] = {"PATH=C:\\TEST", NULL};
```

```
main()
{
    execle("child.exe", "child", "arg1", "arg2", NULL, env);
    /* only get here on an exec error */
    if (errno == ENOENT)
        printf("child.exe not found in current directory\n");
    else if (errno == ENOMEM)
        printf("not enough memory to execute child.exe\n");
    else
        printf("error #%d trying to exec child.exe\n", errno);
}
```

6. **execve() System Function:**

Just like `execle()` you can provide your own environment variables along with `execve()`. You can also pass arguments as arrays as you did in `execv()`.

Syntax:

```
int execve(const char *file, char *const argv[], char *const envp[]);
```

The execve() example can be rewritten as follows:

```
#include <unistd.h>
```

```
int main(void) {
    char *binaryPath = "/bin/bash";
    char *const args[] = {binaryPath, "-c", "echo `Visit $HOSTNAME:$PORT`", NULL};
    char *const env[] = {"HOSTNAME=www.linuxhint.com", "PORT=8080", NULL};

    execve(binaryPath, args, env);

    return 0;
}
```

Linux Exec System Call

- The exec system call is used to execute a file which is residing in an active process. When exec is called the previous executable file is replaced and new file is executed.
- More precisely, we can say that using exec system call will replace the old file or program from the process with a new file or program. The entire content of the process is replaced with a new program.
- The user data segment which executes the exec() system call is replaced with the data file whose name is provided in the argument while calling exec().
- The new program is loaded into the same process space. The current process is just turned into a new process and hence the process id PID is not changed, this is because we are not creating a new process we are just replacing a process with another process in exec.
- If the currently running process contains more than one thread then all the threads will be terminated and the new process image will be loaded and then executed. There are no destructor functions that terminate threads of current process.
- PID of the process is not changed but the data, code, stack, heap, etc. of the process are changed and are replaced with those of newly loaded process. The new process is executed from the entry point.

Exec system call is a collection of functions and in C programming language, the standard names for these functions are as follows:

1. execl
2. execl
3. execlp
4. execv
5. execve
6. execvp

It should be noted here that these functions have the same base *exec* followed by one or more letters. These are explained below:

e: It is an array of pointers that points to environment variables and is passed explicitly to the newly loaded process.

l: l is for the command line arguments passed a list to the function

p: p is the path environment variable which helps to find the file passed as an argument to be loaded into process.

v: v is for the command line arguments. These are passed as an array of pointers to the function.

Why exec is used?

exec is used when the user wants to launch a new file or program in the same process.

Inner Working of exec

Consider the following points to understand the working of exec:

1. Current process image is overwritten with a new process image.
2. New process image is the one you passed as exec argument
3. The currently running process is ended
4. New process image has same process ID, same environment, and same file descriptor (because process is not replaced process image is replaced)
5. The CPU stat and virtual memory is affected. Virtual memory mapping of the current process image is replaced by virtual memory of new process image.

Syntaxes of exec family functions:

The following are the syntaxes for each function of exec:

```
int execl(const char* path, const char* arg, ...)
int execlp(const char* file, const char* arg, ...)
int execlp(const char* path, const char* arg, ..., char* const envp[])
int execv(const char* path, const char* argv[])
int execvp(const char* file, const char* argv[])
int execvp(const char* file, const char* argv[], char *const envp[])
```

Description:

The return type of these functions is Int. When the process image is successfully replaced nothing is returned to calling function because the process that called it is no longer running. But if there is any error -1 will be returned. If any error is occurred an *errno* is set.

In the syntax:

1. **path** is used to specify the full path name of the file which is to be executes.
2. **arg** is the argument passed. It is actually the name of the file which will be executed in the process. Most of the times the value of arg and path is same.
3. **const char* arg** in functions execl(), execlp() and execlp() is considered as arg0, arg1, arg2, ..., argn. It is basically a list of pointers to null terminated strings. Here the first argument points to the filename which will be executed as described in point 2.
4. **envp** is an array which contains pointers that point to the environment variables.
5. **file** is used to specify the path name which will identify the path of new process image file.
6. The functions of exec call that end with **e** are used to change the environment for the new

process image. These functions pass list of environment setting by using the argument **envp**. This argument is an array of characters which points to null terminated String and defines environment variable.

To use the exec family functions, you need to include the following header file in your C program:

```
#include <unistd.h>
```

Example 1: Using exec system call in C program

Consider the following example in which we have used exec system call in C programming in Linux, Ubuntu: We have two c files here example.c and hello.c:

example.c

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    printf("PID of example.c = %d\n", getpid());
    char *args[] = {"Hello", "C", "Programming", NULL};
    execv("./hello", args);
    printf("Back to example.c");
    return 0;
}
```

hello.c

CODE:

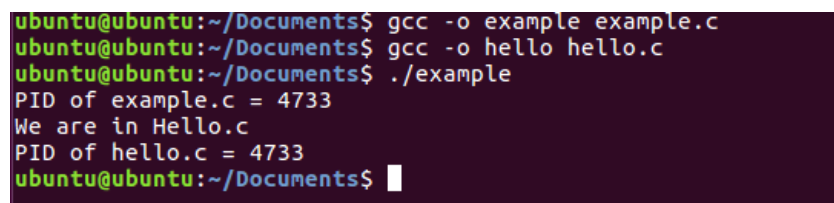
```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    printf("We are in Hello.c\n");
    printf("PID of hello.c = %d\n", getpid());
    return 0;
}
```

OUTPUT:

PID of example.c = 4733

We are in Hello.c

PID of hello.c = 4733



```
ubuntu@ubuntu:~/Documents$ gcc -o example example.c
ubuntu@ubuntu:~/Documents$ gcc -o hello hello.c
ubuntu@ubuntu:~/Documents$ ./example
PID of example.c = 4733
We are in Hello.c
PID of hello.c = 4733
ubuntu@ubuntu:~/Documents$
```

In the above example we have an example.c file and hello.c file. In the example .c file first of all we have printed the ID of the current process (file example.c is running in current process). Then

in the next line we have created an array of character pointers. The last element of this array should be NULL as the terminating point.

Then we have used the function `execv()` which takes the file name and the character pointer array as its argument. It should be noted here that we have used `./` with the name of file, it specifies the path of the file. As the file is in the folder where `example.c` resides so there is no need to specify the full path.

When `execv()` function is called, our process image will be replaced now the file `example.c` is not in the process but the file `hello.c` is in the process. It can be seen that the process ID is same whether `hello.c` is process image or `example.c` is process image because process is same and process image is only replaced.

Then we have another thing to note here which is the `printf()` statement after `execv()` is not executed. This is because control is never returned back to old process image once new process image replaces it. The control only comes back to calling function when replacing process image is unsuccessful. (The return value is -1 in this case).

Lab Tasks

1. **Simulate** suitable command/commands in gedit editor where a child is created to execute a command using `fork()` and `exit()` system calls.
2. **Simulate** suitable command/commands in gedit editor where a child created to execute a command using `wait()` and `exit()` system calls.
3. **Simulate** suitable command/commands in gedit to write equivalent code in C by following the algorithms given

ALGORITHM 1:

STEP 1: Start the program.

STEP 2: Declare fork result as integer.

STEP 3: get process id through function, print parent process with process number.

STEP 4: get process id through function, print process during forking with process number.

STEP 5: fork the process and save it in `forkresult`.

STEP 6: Check `forkresult` is not equal to 0 then get and print child process id through function with fork result, else get and print child process id through function.

STEP 7: get process id through function, print parent process with process number.

STEP 8: Stop the program.

ALGORITHM 2:

STEP 1: Start the program.

STEP 2: Declare `pid` as integer.

STEP 3: Print "hello world".

STEP 4: get process id through function, print "parent process" with process number.

STEP 5: Print statement "before forking".

STEP 6: Fork the process and store it in `pid`.

STEP 7: Print statement "after forking".

STEP 8: Check pid is equal to 0 then get and print “child process” with id through function, else get and print “parent process” with id through function.
STEP 9: Stop the program.

Conclusion:

Lab Instructor
Fasih Ahmad